PlaceNodesコンポーネント群 UI仕様書

1. 概要

1.1 コンポーネントの目的

PlaceNodesは、Petri Net編集画面において、実験ワークフローの「状態（Place）」を視覚的に表現するReactFlowノードコンポーネント群です。Material（材料）、Condition（条件）、Result（結果）の3種類のPlaceノードが存在し、それぞれ異なる色とセマンティクスで実験の状態を表現します。

主な機能:

- Petri Netの状態ノード（Place）の視覚的表現
- フォーマットファイル参照の有無を示すバッジ表示
- テンプレート参照のツールチップ表示
- ReactFlowの入出力ハンドル管理

1.2 使用場面

- **表示タイミング**: Petri Net編集画面（PetriNetEditorInner）でワークフローを表示・編集する際
- **表示条件**: MAIMLファイル内のPlaceノード（material, condition, result）が存在する場合
- **操作**: ReactFlowのノード追加、ドラッグ移動、接続、プロパティ編集

1.3 アクセス権限

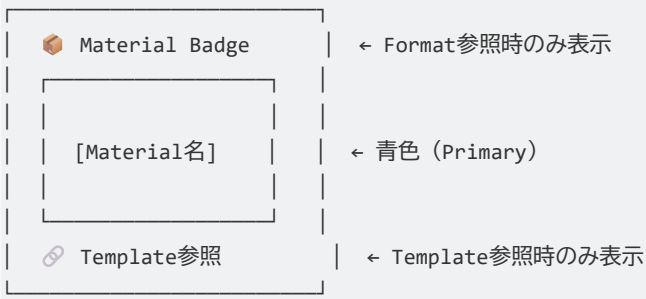
- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: JWT + Session Hybrid認証
- **テナント分離**: organization_idによる分離

---

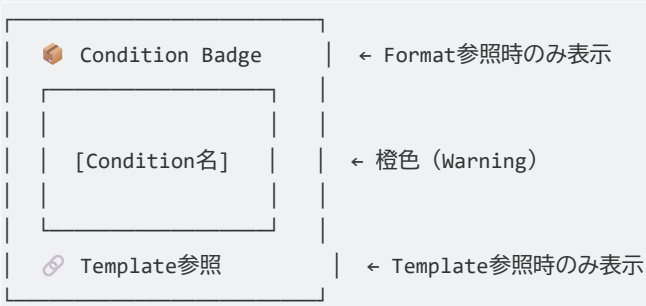
2. 画面構成

2.1 3種類のPlaceノード

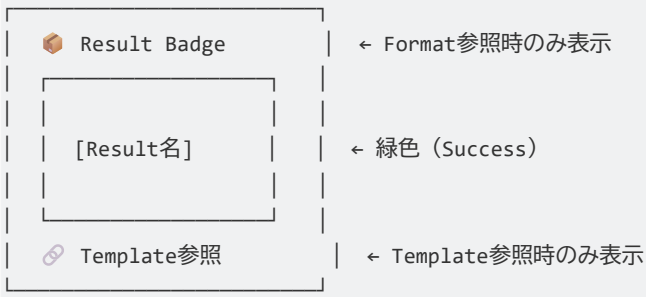
2.1.1 MaterialPlaceNode（材料ノード）



2.1.2 ConditionPlaceNode（条件ノード）



2.1.3 ResultPlaceNode（結果ノード）



2.2 主要UI要素

2.2.1 ノード本体

- **形状**: 円形（borderRadius: 999px）
- **サイズ**: 可変（label長に応じて自動調整）
- **枠線**: 2px solid（選択時: 3px）
- **背景色**:

- Material: `token.colorPrimary`（青）
- Condition: `token.colorWarning`（橙）
- Result: `token.colorSuccess`（緑）

2.2.2 ラベル

- **位置**: ノード中央
- **フォント**: 14px、太字（500）
- **色**: 白（`token.colorWhite`）
- **折り返し**: word-break: break-word
- **余白**: padding: 10px 16px

2.2.3 フォーマットバッジ

- **表示条件**: `isFormatFile === true`
- **テキスト**: "Format"
- **背景色**: `token.colorPrimary`（青）
- **位置**: ノード上部中央（transform: translateX(-50%)）
- **フォントサイズ**: 11px

2.2.4 テンプレート参照インジケータ

- **表示条件**: `templateRef` が存在する場合
- **アイコン**: LinkOutlined（🔗）
- **色**: `token.colorTextSecondary`
- **位置**: ノード下部
- **ツールチップ**: "テンプレート参照: [templateRef]"

2.2.5 ReactFlowハンドル

- **入力ハンドル（上）**:
  - type: "target"
  - position: Position.Top
  - id: `${nodeId}-in`

- **出力ハンドル（下）**：
  - type: "source"
  - position: Position.Bottom
  - id: `${nodeId}-out`

---

3. 機能仕様

3.1 ノードの基本機能

3.1.1 ノードレンダリング

処理フロー:

```
const PlaceNode: React.FC<PlaceNodeData> = ({ data }) => {
  const { token } = theme.useToken();

  // 1. カラーテーマの取得
  const color = getNodeColor(token);

  // 2. 選択状態の判定
  const isSelected = data.selected;

  // 3. ノードスタイルの計算
  const nodeStyle = {
    backgroundColor: color,
    border: `${isSelected ? '3px' : '2px'} solid ${color}`,
    borderRadius: 999,
    // ...
  };

  // 4. JSXレンダリング
  return (
    <>
      {data.isFormatFile && <Badge>Format</Badge>}
      <div style={nodeStyle}>
        {data.label}
      </div>
      {data.templateRef && <TemplateIndicator />}
      <Handle type="target" position={Position.Top} />
      <Handle type="source" position={Position.Bottom} />
    </>
  );
};
```

3.1.2 フォーマットバッジ表示

条件:

- `data.isFormatFile === true`

UI動作:

- ノード上部に "Format" バッジを表示
- 青色の背景、白文字、角丸
- ノード本体の外側に配置（absolute positioning）

3.1.3 テンプレート参照表示

条件:

- `data.templateRef` が存在する（空文字列でない）

UI動作:

- ノード下部にリンクアイコンを表示
- ホバー時にツールチップで参照先を表示
- テキスト: "テンプレート参照: {templateRef}"

3.2 ReactFlow統合

3.2.1 ノードタイプ登録

```
// PetriNetEditorInner.tsxでの登録
const nodeTypes = useMemo(() => ({
  materialPlace: MaterialPlaceNode,
  conditionPlace: ConditionPlaceNode,
  resultPlace: ResultPlaceNode,
  transition: TransitionNode,
}), []);
```

3.2.2 ノードデータ構造

```
interface PlaceNodeData {
  label: string;              // ノード名
```

```
  isFormatFile?: boolean;     // フォーマットファイル参照フラグ
  templateRef?: string;       // テンプレート参照ID
  selected?: boolean;         // 選択状態
  placeType: 'material' | 'condition' | 'result';  // Placeタイプ
}
```

3.3 ノードの相互作用

3.3.1 選択状態

UI動作:

- 選択時: 枠線を3pxに太くする
- 未選択時: 枠線を2pxに戻す
- 選択時の色は変更しない（背景色は維持）

3.3.2 エッジ接続

制約:

- 入力ハンドル（上）: TransitionノードまたはPlaceノードから接続可能
- 出力ハンドル（下）: TransitionノードまたはPlaceノードへ接続可能
- Petri Netの構造制約（Place → Transition → Place）はPetriNetEditorInnerで検証

---

4. データ仕様

4.1 Props定義

4.1.1 MaterialPlaceNode

```
interface MaterialPlaceNodeData {
  label: string;              // Material名
  isFormatFile?: boolean;     // フォーマットファイル参照の有無
  templateRef?: string;       // テンプレート参照ID
  selected?: boolean;         // 選択状態（ReactFlowが管理）
  placeType: 'material';      // 固定値
}
```

```
type MaterialPlaceNodeProps = NodeProps<MaterialPlaceNodeData>;
```

4.1.2 ConditionPlaceNode

```
interface ConditionPlaceNodeData {
  label: string;              // Condition名
  isFormatFile?: boolean;     // フォーマットファイル参照の有無
  templateRef?: string;       // テンプレート参照ID
  selected?: boolean;         // 選択状態（ReactFlowが管理）
  placeType: 'condition';     // 固定値
}

type ConditionPlaceNodeProps = NodeProps<ConditionPlaceNodeData>;
```

4.1.3 ResultPlaceNode

```
interface ResultPlaceNodeData {
  label: string;              // Result名
  isFormatFile?: boolean;     // フォーマットファイル参照の有無
  templateRef?: string;       // テンプレート参照ID
  selected?: boolean;         // 選択状態（ReactFlowが管理）
  placeType: 'result';        // 固定値
}

type ResultPlaceNodeProps = NodeProps<ResultPlaceNodeData>;
```

4.2 Props説明（共通）

Prop名	型	必須	説明
data.label	string	✅	Placeノードの名前（画面中央に表示）
data.isFormatFile	boolean	-	フォーマットファイル参照の有無（trueの場合Formatバッジを表示）
data.templateRef	string	-	テンプレート参照ID（存在する場合リンクアイコンを表示）
data.selected	boolean	-	選択状態（ReactFlowが自動管理、選択時に枠線が太くなる）
data.placeType	'material' | 'condition' | 'result'	✅	Placeノードのタイプ（色の判定に使用）

4.3 スタイル定義

4.3.1 ノードスタイル（共通）

```
const nodeStyle: React.CSSProperties = {
  backgroundColor: color,                          // ノードタイプごとに異なる
  border: `${data.selected ? '3px' : '2px'} solid ${color}`,
  borderRadius: 999,
  padding: '10px 16px',
  minWidth: 120,
  minHeight: 80,
  display: 'flex',
  alignItems: 'center',
  justifyContent: 'center',
  position: 'relative',
  boxShadow: '0 2px 8px rgba(0, 0, 0, 0.15)',
};
```

4.3.2 ラベルスタイル（共通）

```
const labelStyle: React.CSSProperties = {
  color: token.colorWhite,
  fontSize: 14,
  fontWeight: 500,
  textAlign: 'center',
  wordBreak: 'break-word',
  userSelect: 'none',
};
```

4.3.3 フォーマットバッジスタイル（共通）

```
const badgeStyle: React.CSSProperties = {
  position: 'absolute',
  top: -8,
  left: '50%',
  transform: 'translateX(-50%)',
  backgroundColor: token.colorPrimary,
  color: token.colorWhite,
  fontSize: 11,
  padding: '2px 8px',
  borderRadius: 999,
  fontWeight: 600,
  whiteSpace: 'nowrap',
```

```
  boxShadow: '0 1px 4px rgba(0, 0, 0, 0.2)',
};
```

4.3.4 テンプレート参照スタイル（共通）

```
const templateIndicatorStyle: React.CSSProperties = {
  position: 'absolute',
  bottom: -24,
  left: '50%',
  transform: 'translateX(-50%)',
  display: 'flex',
  alignItems: 'center',
  gap: 4,
  color: token.colorTextSecondary,
  fontSize: 12,
  whiteSpace: 'nowrap',
};
```

4.4 カラーテーマ定義

ノードタイプ	背景色	Ant Design Token
MaterialPlace	青	`token.colorPrimary`
ConditionPlace	橙	`token.colorWarning`
ResultPlace	緑	`token.colorSuccess`

カラー取得関数:

```
// MaterialPlaceNode
const color = token.colorPrimary;

// ConditionPlaceNode
const color = token.colorWarning;

// ResultPlaceNode
const color = token.colorSuccess;
```

---

5. API連携

5.1 使用するAPI

PlaceNodesコンポーネント自体はAPIを直接呼び出しません。

データ取得元:

- 親コンポーネント（PetriNetEditorInner）から `data` propsとして受け取る
- MAIMLファイルのパース結果から生成されたReactFlowノードデータ

5.2 データフロー

```
MAIMLファイル
  ↓ パース (maimlParserTyped.ts)
ReactFlowノードデータ生成 (reactFlowConverter.ts)
  ↓
PetriNetEditorInner (nodes state)
  ↓ Props渡し
PlaceNodeコンポーネント (レンダリング)
```

---

6. エラーハンドリング

6.1 エラーケース

PlaceNodesコンポーネントはプレゼンテーション層のため、エラーハンドリングは限定的です。

エラーケース	対処方法
labelが未定義	空文字列を表示（デフォルト動作）
isFormatFileがundefined	falseとして扱う（バッジ非表示）
templateRefが空文字列	リンクアイコン非表示
colorが取得できない	Ant Designのデフォルトカラーを使用

6.2 エラー処理コード

```
// ラベルの安全な表示
const displayLabel = data.label || '';

// フォーマットバッジの表示判定
const showFormatBadge = data.isFormatFile === true;
```

```
// テンプレート参照の表示判定
const showTemplateRef = Boolean(data.templateRef && data.templateRef.trim());
```

---

7. 実装ファイル

7.1 ファイルパス

- **MaterialPlaceNode**: `frontend/src/components/editorv2/components/nodes/MaterialPlaceNode.tsx`
- **ConditionPlaceNode**: `frontend/src/components/editorv2/components/nodes/ConditionPlaceNode.tsx`
- **ResultPlaceNode**: `frontend/src/components/editorv2/components/nodes/ResultPlaceNode.tsx`

7.2 関連ファイル

- **型定義**:
  - `frontend/src/types/maiml-unified.ts` - MAIML型定義
  - `frontend/src/types/editorv2.ts` - エディタ型定義
- **ReactFlow型**: `reactflow` パッケージの `NodeProps`, `Handle`, `Position`
- **コンバーター**: `frontend/src/utils/maiml/reactFlowConverter.ts` - MAIMLからReactFlowノードへの変換

7.3 依存コンポーネント

外部ライブラリ:

- `reactflow` - ReactFlowの `Handle` コンポーネント
- `antd` - `theme.useToken()` でデザイントークンを取得
- `@ant-design/icons` - `LinkOutlined` アイコン

ReactFlowハンドル:

- `<Handle type="target" position={Position.Top} id={nodeId-in} />` - 入力ハンドル
- `<Handle type="source" position={Position.Bottom} id={nodeId-out} />` - 出力ハンドル

7.4 親コンポーネント

- **PetriNetEditorInner.tsx** - ReactFlowエディタのメインコンポーネント
  - `nodeTypes` に3つのPlaceNodeを登録
  - `nodes` stateでPlaceノードのデータを管理
  - Placeノードの追加、削除、更新を行う

---

8. 補足情報

8.1 実装の共通点

3つのPlaceNodeコンポーネントは以下の点で共通しています：

1. **Props構造**: 同一のインターフェース（placeTypeのみ異なる）
2. **レンダリングロジック**: ほぼ同一（色の取得のみ異なる）
3. **スタイル定義**: 完全に同一
4. **ハンドル配置**: 完全に同一（上:入力、下:出力）

8.2 実装の相違点

唯一の相違点は**カラーテーマ**です：

```
// MaterialPlaceNode.tsx
const color = token.colorPrimary;  // 青

// ConditionPlaceNode.tsx
const color = token.colorWarning;  // 橙

// ResultPlaceNode.tsx
const color = token.colorSuccess;  // 緑
```

8.3 リファクタリングの可能性

将来的には、以下のようなリファクタリングが可能です：

```
// 統合案（共通ベースコンポーネント）
const PlaceNode: React.FC<PlaceNodeProps> = ({ data }) => {
  const colorMap = {
    material: token.colorPrimary,
    condition: token.colorWarning,
    result: token.colorSuccess,
  };

  const color = colorMap[data.placeType];

  // 共通のレンダリングロジック
};

// ReactFlowノードタイプ登録
const nodeTypes = {
  materialPlace: (props) => <PlaceNode {...props} data={{ ...props.data, placeType: 'material' }} />,
  conditionPlace: (props) => <PlaceNode {...props} data={{ ...props.data, placeType: 'condition' }} />,
  resultPlace: (props) => <PlaceNode {...props} data={{ ...props.data, placeType: 'result' }} />,
};
```

8.4 Petri Netでの役割

Material（材料）：

- 実験に使用する材料・試料の状態
- 例:「原料A」「試料B」「サンプルC」

Condition（条件）：

- 実験条件・パラメータの状態
- 例:「温度設定」「圧力条件」「時間」

Result（結果）：

- 実験結果・成果物の状態
- 例:「測定データ」「生成物」「分析結果」

8.5 使用例（ReactFlowでのノード追加）

```
// PetriNetEditorInnerでのノード追加例
const newNode: Node = {
  id: `material-${Date.now()}`,
  type: 'materialPlace',  // ← MaterialPlaceNodeを指定
  position: { x: 100, y: 100 },
  data: {
    label: '原料A',
    isFormatFile: true,
    templateRef: 'template-123',
    placeType: 'material',
  },
};

setNodes((nds) => [...nds, newNode]);
```

---

9. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで3つのPlaceNodeを統合したUI仕様書を作成	Cline